

Министерство науки и высшего образования Российской Федерации
Образовательная организация

КУРСОВАЯ РАБОТА

по дисциплине: программная инженерия и интеллектуальные системы

**Исследование адаптивной оркестрации многоагентных LLM-систем на основе графов
зависимостей задач**

Выполнил: студент
Проверил: преподаватель

Оглавление

Введение	3
1 Теоретические основы	5
1.1 Многоагентные LLM-системы	5
1.2 DAG-графы задач	5
1.3 Topology execution strategies	5
1.4 Critical path и weighted critical path	6
1.5 Trade-off latency, cost, quality и robustness	6
2 Проектирование и архитектура системы	7
2.1 Общая архитектура	7
2.2 Модули проекта	7
2.3 DAG execution pipeline	8
3 Реализация	9
3.1 Node, Edge и DAG	9
3.2 GraphMetrics и weighted metrics	9
3.3 Topology selectors	10
3.4 Executors и mock agents	10
3.5 Quality и robustness evaluation	10
3.6 Benchmark runner, CLI и visualization	10
4 Экспериментальное исследование	11
4.1 Synthetic graph scenarios	11
4.2 Сравнимые стратегии	11
4.3 Benchmark methodology	11
4.4 Collected metrics	12
5 Анализ результатов	13
5.1 Benchmark summary	13
5.2 Comparison of topology strategies	14
5.3 Comparison of selector modes	14
5.4 Визуализации	15

5.5 Ограничения результатов	15
Заключение	17
А Команды воспроизведения	19
В Файлы результатов	20

Введение

Современные LLM-системы всё чаще строятся не как одиночный вызов языковой модели, а как композиция нескольких специализированных агентов. Один агент может планировать, другой выполнять исследование, третий проверять результат, а финальный компонент синтезировать ответ. Такая архитектура повышает выразительность системы, но создаёт новую инженерную задачу: необходимо выбрать порядок и топологию исполнения подзадач.

Актуальность работы связана с тем, что простая последовательная оркестрация плохо использует независимость подзадач, а наивный параллелизм может добавлять лишние накладные расходы. Для реалистичного сравнения topology strategies недостаточно измерять только latency: необходимо учитывать стоимость узлов, critical path, качество результата и устойчивость к отказам.

Цель работы --- разработать и исследовать прототип адаптивной оркестрации многоагентных LLM-систем, в котором задача представляется DAG-графом, а топология исполнения выбирается на основе структурных, стоимостных и экспериментально полученных признаков.

Для достижения цели были поставлены следующие задачи:

1. реализовать модель DAG с узлами, рёбрами и зависимостями;
2. реализовать structural и weighted graph metrics;
3. реализовать static и adaptive topology selectors;
4. добавить cost-aware и learned adaptive selection;
5. реализовать execution layer с mock agents и tracing;
6. добавить quality evaluation и failure simulation;
7. построить benchmark runner, CLI и конфигурации экспериментов;
8. проверить проект unit tests и reproducibility checks;
9. оформить результаты в виде CSV, графиков и timeline visualization.

Объект исследования --- многоагентные LLM-системы, исполняющие сложные задачи как набор зависимых подзадач.

Предмет исследования --- методы выбора topology execution strategy для DAG-задач с учётом latency, cost, quality и robustness.

Методы исследования включают моделирование DAG, анализ критического пути, synthetic benchmarks, эвристический scoring, lightweight nearest-neighbor learned baseline, unit testing и анализ execution traces.

Практическая значимость работы состоит в создании воспроизводимого прототипа **adaptive-mas-coursework**, который позволяет сравнивать orchestration strategies локально без внешних API, сохранять результаты в CSV и строить визуализации для анализа расписания выполнения DAG.

Глава 1

Теоретические основы

1.1. Многоагентные LLM-системы

Многоагентная LLM-система представляет собой программную архитектуру, в которой сложная задача распределяется между несколькими агентами. Общие принципы анализа multi-agent systems описаны в классической литературе по многоагентным системам [6]. В реальных системах агенты могут обращаться к языковой модели, инструментам поиска, базам знаний или внешним API. В реализованном проекте используются mock agents, поэтому исследование сфокусировано на orchestration layer, а не на качестве конкретного LLM backend.

Основное преимущество многоагентного подхода заключается в декомпозиции: разные шаги могут выполняться независимо или частично параллельно. Однако это преимущество проявляется только при корректном управлении зависимостями.

1.2. DAG-графы задач

Задача моделируется направленным ациклическим графом $G = (V, E)$, где V --- множество подзадач, а E --- зависимости между ними. Ребро $u \rightarrow v$ означает, что узел v может быть выполнен только после завершения узла u . Ациклическость важна, потому что цикл зависимостей привёл бы к невозможности построить корректный порядок исполнения.

Для DAG существует топологическая сортировка. Она задаёт допустимый порядок исполнения узлов и служит основой для sequential, parallel, hierarchical и hybrid executors. Алгоритмическая база топологической сортировки и работы с графами опирается на стандартные методы анализа графов [4, 2].

1.3. Topology execution strategies

В проекте рассматриваются четыре базовые топологии:

Таблица 1.1: Topology execution strategies

Topology	Описание
<code>sequential</code>	Узлы выполняются по одному в топологическом порядке; overhead минимален, параллелизм не используется.
<code>parallel</code>	На каждом шаге запускаются все готовые узлы, зависимости которых выполнены.
<code>hierarchical</code>	Исполнение организуется через coordinator/aggregate pattern с дополнительными координационными затратами.
<code>hybrid</code>	Граф выполняется слоями: внутри слоя возможен параллелизм, между слоями сохраняется порядок.

1.4. Critical path и weighted critical path

Critical path --- самый длинный путь зависимостей в DAG. Он задаёт нижнюю границу времени исполнения: узлы на одном пути не могут быть выполнены параллельно относительно друг друга. Идея critical path широко используется в планировании и управлении проектами [5].

В weighted DAG каждому узлу v сопоставляется стоимость $c(v)$. В проекте она берётся из `node.data["cost"]`, а если стоимость не задана, используется безопасное значение по умолчанию `DEFAULT_NODE_COST = 0.1`. Weighted critical path определяется как:

$$WCP(G) = \max_{p \in Paths(G)} \sum_{v \in p} c(v).$$

Эта метрика точнее обычной длины пути, потому что учитывает неоднородность узлов: короткий путь из дорогих задач может сильнее влиять на latency, чем длинный путь из дешёвых задач.

1.5. Trade-off latency, cost, quality и robustness

Выбор topology является многокритериальной задачей. Parallel execution может снижать latency, но увеличивать coordination overhead. Fallback logic может повышать success rate, но ухудшать consistency score. Learned selector может повторять удачные решения из synthetic benchmark data, но его качество зависит от training set и objective score.

В работе используются следующие группы метрик:

1. latency и execution cost;
2. structural и weighted graph metrics;
3. quality metrics;
4. robustness metrics;
5. selected topology и selector mode.

Глава 2

Проектирование и архитектура системы

2.1. Общая архитектура

Проект `adaptive-mas-coursework` организован как набор модулей Python и использует `networkx.DiGraph` как базовую структуру для DAG [3, 1]. Архитектура показана на рисунке 2.1.

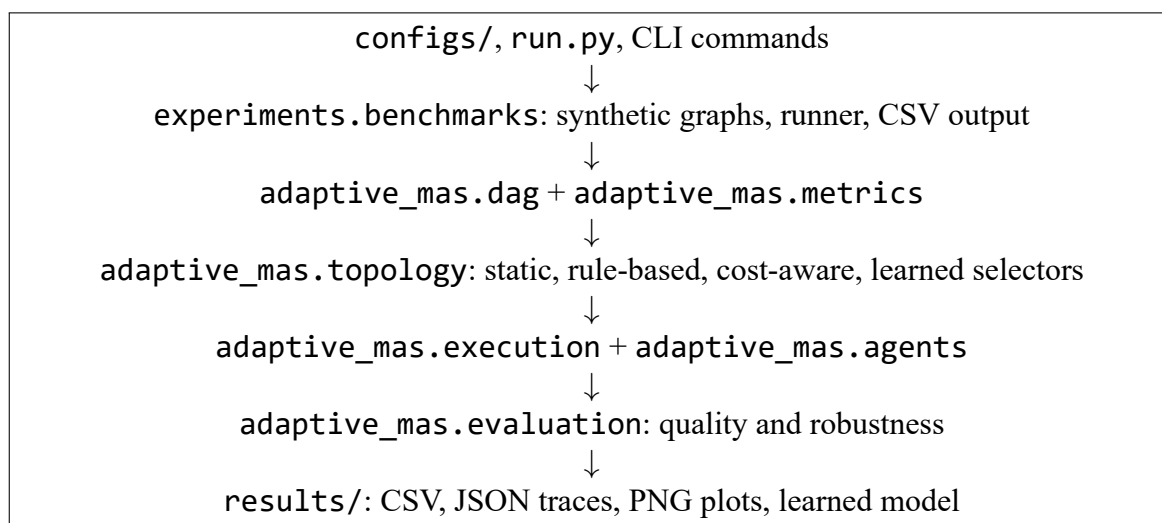


Рис. 2.1: Архитектурная схема проекта

2.2. Модули проекта

Таблица 2.1: Реальные модули проекта

Модуль	Назначение
<code>adaptive_mas.dag</code>	Node, Edge, DAG, зависимости и топологическая сортировка.

Модуль	Назначение
<code>adaptive_mas.metrics</code>	Structural metrics, weighted metrics, critical path и weighted critical path.
<code>adaptive_mas.topology</code>	TopologyType, rule-based selector, cost-aware selector, learned selector.
<code>adaptive_mas.execution</code>	Sequential, parallel, hierarchical и hybrid executors, execution traces, failure handling.
<code>adaptive_mas.agents</code>	Mock agents, имитирующие выполнение ролей без real LLM API.
<code>adaptive_mas.evaluation</code>	Heuristic quality evaluation и robustness metrics.
<code>experiments.benchmarks</code>	Synthetic graph factory и benchmark runner.
<code>run.py</code>	Единый CLI для demo, benchmark, visualization, experiments и training learned selector.
<code>configs/</code>	JSON-конфигурации экспериментов.
<code>results/</code>	CSV, PNG, JSON traces и learned selector model.

2.3. DAG execution pipeline

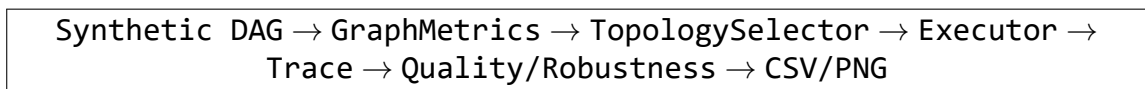


Рис. 2.2: Pipeline исполнения DAG-задачи

Глава 3

Реализация

3.1. Node, Edge и DAG

Классы `Node`, `Edge` и `DAG` находятся в `adaptive_mas.dag`. Узел хранит идентификатор, `task`, `agent` и произвольные данные, включая возможную стоимость `node.data["cost"]`. Рёбра описывают зависимости. `DAG` предоставляет методы добавления узлов и рёбер, проверки ацикличности, получения `predecessors/successors` и топологической сортировки.

3.2. GraphMetrics и weighted metrics

`GraphMetrics` рассчитывает `depth`, `density`, `parallel width`, `max degree` и `critical path length`. В финальной версии добавлены `weighted metrics`:

1. `node_cost(node_id);`
2. `total_node_cost();`
3. `average_node_cost();`
4. `max_node_cost();`
5. `cost_variance();`
6. `weighted_critical_path_length();`
7. `weighted_parallel_width();`

Если стоимость узла отсутствует, используется `default mock execution cost`. Метрики безопасно обрабатывают пустой `DAG`.

3.3. Topology selectors

Rule-based adaptive selector выбирает topology по структурным эвристикам: глубина, ширина, плотность и max degree. Cost-aware selector оценивает все поддерживаемые топологии и использует простую интерпретируемую формулу:

$$score = expected\ latency + coordination\ overhead + critical\ path\ penalty.$$

Learned adaptive selector реализован в `adaptive_mas.topology.learned_selector`. Это lightweight nearest-neighbor baseline без тяжёлых ML-зависимостей. Feature vector включает node count, edge count, graph depth, density, parallel width, max degree, total node cost, average node cost, max node cost, weighted critical path и weighted parallel width.

Training objective documented in the model file:

$$objective_score = estimated_latency_with_overhead + 0.05 \cdot execution_cost.$$

3.4. Executors и mock agents

В execution layer реализованы `SequentialExecutor`, `ParallelExecutor`, `HierarchicalExecutor` и `HybridExecutor`. Executors собирают execution trace: `node_id`, task, agent, start time, end time, duration, topology, status, retry count и fallback flags.

Mock agents имитируют роли planner, researcher, critic и synthesizer. Они не обращаются к LLM API, что делает эксперименты быстрыми и воспроизводимыми.

3.5. Quality и robustness evaluation

`QualityEvaluator` реализует deterministic heuristic model. Он оценивает completeness, consistency, synthesis, dependency coverage и overall quality. Failed, skipped и fallback nodes снижают соответствующие scores.

Robustness evaluation считает success rate, failed node count, skipped node count, retry count total, fallback count, recovery success rate и wasted work estimate. Failure simulation поддерживает synthetic exception, timeout и empty result.

3.6. Benchmark runner, CLI и visualization

Benchmark runner генерирует synthetic graphs, запускает стратегии и сохраняет результаты в CSV. CLI `run.py` поддерживает команды `demo`, `benchmark`, `visualize`, `experiment` и `train-selector`. Timeline visualization строит Gantt chart из execution trace и сохраняет PNG в `results/`.

Глава 4

Экспериментальное исследование

4.1. Synthetic graph scenarios

В финальном benchmark использованы четыре сценария:

1. `wide_sparse`;
2. `deep_dependency`;
3. `layered`;
4. `centralized_coordinator`.

Они покрывают широкий параллелизм, глубокие зависимости, послойную структуру и coordinator pattern.

4.2. Сравнимые стратегии

Таблица 4.1: Selector modes

Mode	Смысл
<code>static</code>	Исполняется явно заданная topology: sequential, parallel, hierarchical или hybrid.
<code>rule_based_adaptive</code>	Выбор topology по structural graph metrics.
<code>cost_aware_adaptive</code>	Выбор topology по latency, overhead и critical path penalty.
<code>learned_adaptive</code>	Nearest-neighbor baseline, обученный на synthetic benchmark data.

4.3. Benchmark methodology

Финальный запуск был выполнен командой:

```
python run.py benchmark --selector all --model results/learned_selector_model.json --runs 10
```

Перед этим learned selector был обучен:

```
python run.py train-selector --runs 100 --output results/learned_selector_model.json
```

Результаты сохранены в `results/benchmark_results.csv` и `results/benchmark_summary.json`.
Файл `results/experiment_config_used.json` фиксирует использованную конфигурацию. Unit tests запускались командой:

```
$env:PYTHONDONTWRITEBYTECODE='1'; python -m unittest discover -s tests
```

Фактический результат тестирования: 56 тестов, статус ОК.

4.4. Collected metrics

Таблица 4.2: Основные метрики benchmark

Метрика	Описание
latency	Итоговая задержка исполнения DAG.
cost	Synthetic execution cost.
coordination_overhead	Накладные расходы координации.
parallel_efficiency	Эффективность относительно critical path.
weighted_critical_path	Максимальная суммарная стоимость пути в DAG.
weighted_parallel_width	Максимальная суммарная стоимость одного уровня.
overall_quality_score	Итоговая heuristic quality score.
success_rate	Доля успешно выполненных узлов.
selector_mode	Static, rule-based, cost-aware или learned adaptive.
selected_topology	Фактически выбранная topology.

Глава 5

Анализ результатов

5.1. Benchmark summary

В таблице 5.1 приведены агрегированные результаты из фактического файла `results/benchmark_summary.csv` сформированного финальным запуском. Все значения являются результатом текущего состояния репозитория; при изменении seed, числа runs или реализации они должны пересчитываться.

Таблица 5.1: Benchmark summary из `results/benchmark_summary.csv`

Graph type	Mode	Topology	Runs	Avg latency	Avg cost
centralized_coordinator	cost-aware	parallel	10	1.119	4.100
centralized_coordinator	learned	parallel	10	1.119	4.100
centralized_coordinator	rule-based	hierarchical	10	1.238	3.984
centralized_coordinator	static	hierarchical	10	1.238	3.984
centralized_coordinator	static	hybrid	10	1.265	3.959
centralized_coordinator	static	parallel	10	1.119	4.100
centralized_coordinator	static	sequential	10	3.701	3.666
deep_dependency	cost-aware	sequential	10	3.697	3.664
deep_dependency	learned	sequential	10	3.697	3.664
deep_dependency	rule-based	sequential	10	3.697	3.664
deep_dependency	static	hierarchical	10	4.158	4.213
deep_dependency	static	hybrid	10	4.053	4.031
deep_dependency	static	parallel	10	3.869	3.971
deep_dependency	static	sequential	10	3.697	3.664
layered	cost-aware	parallel	10	0.919	3.928
layered	learned	parallel	10	0.919	3.928
layered	rule-based	hybrid	10	1.154	3.929
layered	static	hierarchical	10	1.207	4.036

Graph type	Mode	Topology	Runs	Avg latency	Avg cost
layered	static	hybrid	10	1.154	3.929
layered	static	parallel	10	0.919	3.928
layered	static	sequential	10	3.700	3.665
wide_sparse	cost-aware	parallel	10	1.257	4.045
wide_sparse	learned	parallel	10	1.257	4.045
wide_sparse	rule-based	parallel	10	1.257	4.045
wide_sparse	static	hierarchical	10	1.397	3.991
wide_sparse	static	hybrid	10	1.377	3.920
wide_sparse	static	parallel	10	1.257	4.045
wide_sparse	static	sequential	10	3.683	3.655

5.2. Comparison of topology strategies

Если агрегировать только static strategies по четырём graph types, текущий CSV даёт значения из таблицы 5.2. Эти числа показывают, что в synthetic setup parallel имеет меньшую среднюю latency, чем другие static режимы, но также имеет более высокую среднюю стоимость, чем sequential. Это не доказывает универсальное превосходство parallel topology, потому что результаты зависят от набора synthetic graphs и mock cost model.

Таблица 5.2: Средние значения static topologies по четырём graph types

Topology	Avg latency	Avg cost	Avg efficiency
hierarchical	2.000	4.056	0.633
hybrid	1.962	3.960	0.640
parallel	1.791	4.011	0.729
sequential	3.695	3.663	0.387

5.3. Comparison of selector modes

Таблица 5.3: Средние значения selector modes по текущему benchmark summary

Mode	Groups	Avg latency	Avg cost	Avg quality
cost-aware adaptive	4	1.748	3.934	0.900
learned adaptive	4	1.748	3.934	0.900
rule-based adaptive	4	1.836	3.905	0.900
static	16	2.362	3.922	0.900

В текущем финальном запуске cost-aware и learned selectors выбрали одинаковые topology для всех четырёх graph categories. Это является фактическим наблюдением по сохранённому CSV, но не означает, что learned selector всегда будет совпадать с cost-aware selector или

превосходить другие режимы. Learned selector обучен на synthetic benchmark data и является baseline.

5.4. Визуализации

Файлы визуализаций находятся в `results/`. В работу включены реальные PNG, созданные проектом.

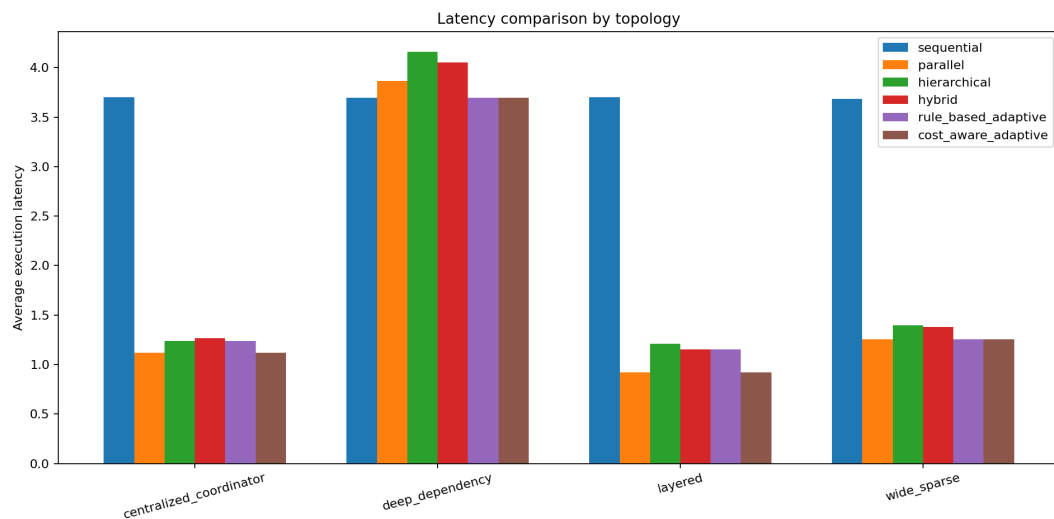


Рис. 5.1: Сравнение latency по graph type и topology

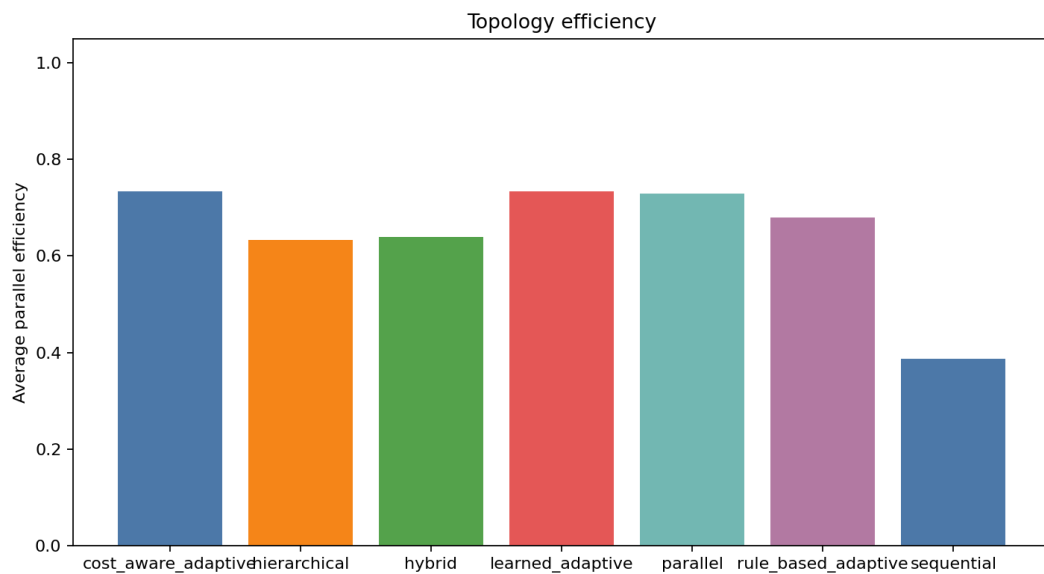


Рис. 5.2: Parallel efficiency topology strategies

5.5. Ограничения результатов

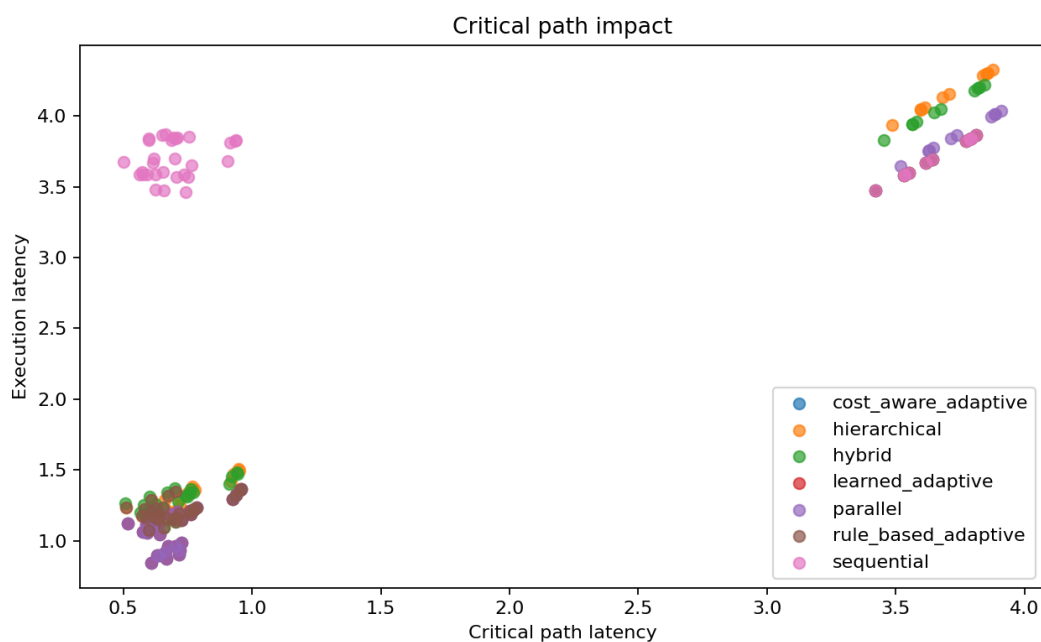


Рис. 5.3: Влияние critical path latency на total latency

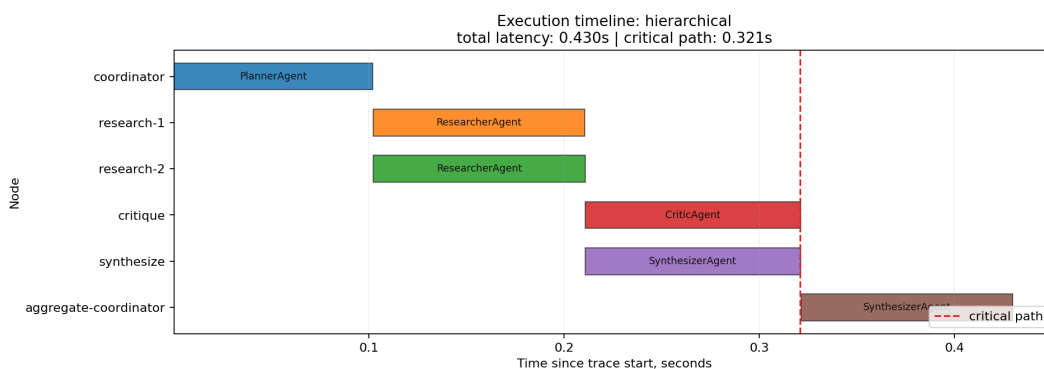


Рис. 5.4: Gantt chart / execution timeline из results/execution_trace.json

Таблица 5.4: Ограничения эксперимента

Ограничение	Влияние на интерпретацию
Mock agents	Latency и results не являются измерениями real LLM API.
Synthetic graphs	Scenarios упрощают реальные workflows и dependency patterns.
Heuristic quality model	Quality score не измеряет семантическое качество текста.
Synthetic failure model	Failure probabilities не откалиброваны на production traces.
Learned baseline	Nearest-neighbor selector не является полноценной ML policy.
Fixed benchmark configuration	Численные результаты относятся к текущему seed/runs/config и должны пересчитываться при изменениях.

Заключение

В ходе работы был реализован и стабилизирован исследовательский прототип адаптивной оркестрации многоагентных LLM-систем на основе DAG. Проект включает DAG model, graph metrics, weighted metrics, topology selectors, executors, mock agents, quality evaluation, failure simulation, robustness metrics, benchmark runner, CLI/config layer, visualizations и tests.

Ключевые улучшения финальной версии:

1. cost-aware adaptive selector;
2. weighted graph metrics и weighted critical path;
3. learned adaptive selector как lightweight nearest-neighbor baseline;
4. Gantt chart / execution timeline visualization;
5. heuristic quality evaluation;
6. synthetic failure simulation and recovery logic;
7. unified CLI и experiment configs;
8. unit tests и reproducibility checks.

По фактическому `benchmark_summary.csv` финальный benchmark сравнил static topologies, rule-based adaptive, cost-aware adaptive и learned adaptive. В текущем запуске learned selector использовал сохранённую модель `results/learned_selector_model.json`. Полученные результаты показывают, что учет стоимости и critical path помогает выбирать более подходящую topology в synthetic setup, однако работа не утверждает универсальное превосходство какого-либо selector mode.

Основные ограничения остаются следующими: используются mock agents, synthetic graphs, heuristic quality evaluation и synthetic robustness model; learned selector является baseline и требует калибровки на реальных traces.

Future work:

1. подключить real LLM backend;
2. откалибровать cost, overhead и failure model на реальных traces;
3. развить learned selector до более сложной learned policy;

4. добавить dynamic re-orchestration во время выполнения DAG;
5. провести evaluation на реальных задачах и real-world traces.

Приложение А

Команды воспроизведения

```
$env:PYTHONDONTWRITEBYTECODE='1'; python -m unittest discover -s tests
python demo.py
python run.py benchmark --runs 1
python run.py train-selector --runs 100 --output results/
    learned_selector_model.json
python run.py benchmark --selector learned_adaptive --model results/
    learned_selector_model.json --runs 10
python run.py benchmark --selector all --model results/learned_selector_model
    .json --runs 10
python run.py visualize --trace results/execution_trace.json
```

Приложение В

Файлы результатов

В работе использованы следующие фактические файлы:

1. `results/benchmark_results.csv`;
2. `results/benchmark_summary.csv`;
3. `results/learned_selector_model.json`;
4. `results/latency_comparison.png`;
5. `results/topology_efficiency.png`;
6. `results/critical_path_impact.png`;
7. `results/timeline_execution_trace_hierarchical.png`.

Литература

- [1]almazgde. adaptive-mas-coursework, 2026. Local project repository used as the implementation basis for this coursework.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 2009.
- [3] NetworkX Developers. Networkx documentation, 2026.
- [4] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558--562, 1962.
- [5] Harold Kerzner. *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*. Wiley, 2017.
- [6] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley and Sons, 2009.